PNAS PERSPECTIVE OPEN ACCESS

The principles behind equivariant neural networks for physics and chemistry

Risi Kondor[a,b,1]

Edited by Pablo Debenedetti, Princeton University, Princeton, NJ; received October 29, 2024; accepted July 13, 2025

A distinguishing feature of the neural network models used in Physics and Chemistry is that they must obey basic underlying symmetries, such as symmetry to translations, rotations, and the exchange of identical particles. Over the course of the last several years, the artificial neural networks community has developed a class of networks called group-equivariant neural nets that can efficiently "bake-in" such symmetries into the structure of the network itself. Equivariant neural nets leverage ideas from group representation theory and express all variables in the generalized Fourier space corresponding to the underlying group. In this article, we review this formalism and derive the general form of operations allowable in equivariant neural networks. Specifically, we discuss why the Clebsch–Gordan transform appears in such architectures, and how it can play the role of an equivariant nonlinearity.

equivariant neural networks | neural networks | Clebsch–Gordan product | noncommutative harmonic analysis

Much of today's AI research focuses on developing general-purpose systems, such as large language models, trained with injecting as little domain specific prior information as possible. In the case of machine learning for science, however, the situation is somewhat different. In this article, we argue that certain fundamental physical constraints, particularly symmetries (invariances), should in fact be explicitly incorporated into AI/ML models for Physics and Chemistry:

1. Compared to some of the paradigmatic large scale commercial applications of AI (image recognition, LLMs, etc.) empirical data in typical Physics/Chemistry applications is not as plentiful and considerably more expensive to generate on demand from experiments or ab initio simulations.
2. In contrast to, for example, the rules of English grammar, invariances can be precisely described mathematically, and we expect physical systems to obey invariances not just in an approximate sense, but exactly.
3. Even if it performs well on some specific learning objective, a learning algorithm that does not hard code such fundamental laws might not generalize well to out-of-sample data or might produce predictions that are not physically feasible. For example, a machine learned force field that takes no account of translation and rotation symmetry might perform acceptably well on static benchmarks, but produce trajectories that violate the conservation of angular momentum or energy when incorporated in a molecular dynamics simulation.

From a pragmatic point of view, the question is what is the extra cost of constraining models to respect invariances, both in terms of raw per-iteration computation and possibly the algorithm's ease of trainability. Fortunately, for artificial neural networks (ANNs), we now have a comprehensive formalism called Equivariant Neural Networks that can efficiently implement symmetries to a wide range of groups, from the three-dimensional rotation group SO(3) through noncompact groups such as the Lorentz group O(3, 1) to finite groups such as the symmetric group $\mathbb{S}_n$ of permutations. This formalism is what we focus on in the present paper.

Similarly to how symmetries are reflected in quantum mechanics, the theory of equivariant neural networks is based on group representation theory. As we explain in this article, the power of representation theory is that it provides a way of reducing abstract problems on noncommutative groups to effectively just linear algebra. Since modern ANNs are built out of similar matrix and tensor operations, representation theory makes it relatively easy to combine equivariance with any of today's standard neural network architectures, from classical convolutional type feed-forward models, through transformers, to other generative models. In addition, because the formalism is similar in structure to that of actual physical laws, the models learned with equivariant neural networks can be easier to interpret and integrate with other numerical algorithms.

1.1. Scope. Since the introduction of the term in a sequence of seminal papers (1–4), the field of equivariant neural networks has become a prolific field that researchers from Signal Processing, Computer Vision, Graphics, Robotics, Applied Mathematics, Chemistry, and Physics have all contributed to. In this Perspective, we focus on equivariant architectures for *point cloud data*, i.e., collections of particles embedded in Euclidean space, because this is what is most

Author affiliations: [a]Department of Computer Science, The University of Chicago, Chicago, IL 60605; and [b]Department of Statistics, Committee on Computational and Applied Mathematics, The University of Chicago, Chicago, IL 60605

Author contributions: R.K. wrote the paper.

Competing interest statement: R.K. is an inventor on US patent number 11,934,478 on Spherical convolutional neural networks that is related to the subject matter of this article. R.K. is the founder of Intertwiner Inc., which develops technologies related to equivariant neural networks.

This article is a PNAS Direct Submission.

Copyright © 2025 the Author(s). Published by PNAS. This open access article is distributed under Creative Commons Attribution-NonCommercial-NoDerivatives License 4.0 (CC BY-NC-ND).

[1]Email: risi@cs.uchicago.edu.

Published October 6, 2025.

Downloaded from https://www.pnas.org by 148.252.133.20 on April 23, 2026 from IP address 148.252.133.20.

directly relevant to molecular systems. On the Computer Science side, this topic is often categorized as a part of Geometric Deep Learning (5). However, some of the core ideas, in particular, the use of group representations to capture symmetries in neural networks, go back much farther in the literature (6).

Rather than highlighting individual architectures or necessarily following the historical development of the field, we aim to discuss the underlying mathematical principles, and show the connections between different approaches.

1.2. Overview. Our exposition is structured as follows. Section 2 discusses why equivariance arises as a natural requirement in modeling physical systems, how it is related to invariance, and what it means at the level of individual "neurons."

The purpose of Section 3 is to introduce the mathematical machinery needed to construct equivariant networks, specifically the concept of irreducible representations and a form of generalized (noncommutative) harmonic analysis. Our goal is to introduce these mathematical concepts with the minimum of technical complications and to show that they are so intimately linked to how groups act on vector spaces that their appearance in any discussion of symmetries is almost inevitable.

The core results described in our article appear in Section 4, where we give an explicit set of recipes for constructing equivariant neurons. One particular aspect that we focus on is the appearance of the so-called Clebsch–Gordan product, and how it can serve as an equivariant nonlinearity.

Finally, in Section 5, we briefly discuss how the equivariant neural network formalism is related to neural networks on graphs, which are also important for Physics and Chemistry.

2. Equivariance in Neural Networks

While not necessarily described in exactly these terms, the idea of equivariance in neural networks goes back to Convolutional Neural Networks (CNNs), one of the most iconic neural architectures, widely used in image recognition since the 1980s (7). The basic idea of CNNs is to learn a certain number of small image filters w, each represented as an $n \times n$ array of real numbers, and match them against the input h at every possible location, followed by applying a pointwise nonlinearity σ, called the *activation function*:

$$h^{\text{out}}(x,y) = \sigma\Big(\sum_{x'=1}^{n}\sum_{y'=1}^{n} w(x',y')\, h^{\text{in}}(x+x', y+y')\Big). \qquad [1]$$

Formally, the expression in parentheses is the *cross-correlation* of h with w, but in the neural networks community, it is often referred to as *convolution*, which is a closely related concept. As for the activation function, a popular choice today is the so-called Rectified Linear Unit (ReLU) $\sigma_{\text{ReLU}}(z) = \max(0, z)$, but many other choices are possible.

Practical CNNs have not just one, but multiple convolutional layers, the output of each layer feeding into the next. In addition, each layer usually comprises several different filters, resulting in multiple *channels*. The most salient feature of CNNs however is that they reflect a fundamental symmetry of natural images: Because [**1**] prescribes that exactly the same filter or filters be applied at every pixel location, CNNs can detect the same visual features everywhere in the image plane (Fig. 1). An equivalent way of expressing this is that if we translate the input image h^{in} by an integer amount $\boldsymbol{t} = (t_x, t_y)$,

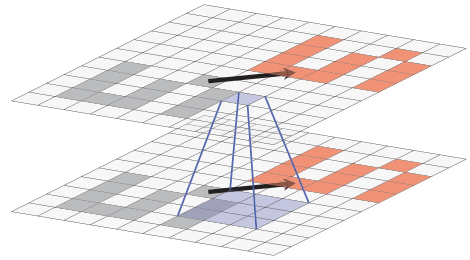

Fig. 1. Classical convolutional neural networks are constructed in such a way that if the input image is translated by an integer vector $\boldsymbol{t}$, then (barring complications such as pooling, etc.) the output of each downstream layer will transform the same way. Equivariant neural networks generalize this behavior to the action of other groups, including noncommutative ones, such as the group SO(3) of rotations in 3D.

$$(h^{\text{in}})'(x,y) = h^{\text{in}}(x - t_x,\, y - t_y),$$

then the output h^ℓ of each subsequent convolutional layer will translate the same way:

$$h^\ell \longmapsto (h^\ell)', \qquad (h^\ell)'(x,y) = h^\ell(x - t_x,\, y - t_y).$$

Equivariant neural networks in Physics and Chemistry are a generalization of this type of architecture from planar translations to other symmetry groups.

A typical use case for equivariant networks is learning some property of a molecular system composed of N atoms with position vectors $\xi_1, \ldots, \xi_N$ (Fig. 2). This scenario is already subject to three distinct symmetries:

1. Symmetry to translations:

$$\xi_i \longmapsto \xi_i - \boldsymbol{t} \qquad i = 1, 2, \ldots, N$$

for any translation vector $\boldsymbol{t} \in \mathbb{R}^3$.

2. Symmetry to rotations:

$$\xi_i \longmapsto \boldsymbol{R}\,\xi_i \qquad i = 1, 2, \ldots, N$$

for any rotation matrix $\boldsymbol{R} \in \mathbb{R}^{3\times 3}$.

3. Symmetry to exchange of particles:

$$\xi_i \longmapsto \xi_{\sigma(i)} \qquad i = 1, 2, \ldots, N,$$

where, in the case of identical atoms, σ can be any permutation of $\{1, 2, \ldots, N\}$.

Before describing how equivariant networks can account for these symmetries, we start by sketching a very general picture of what constitutes a modern neural network.

Downloaded from https://www.pnas.org by 148.252.133.20 on April 23, 2026 from IP address 148.252.133.20.

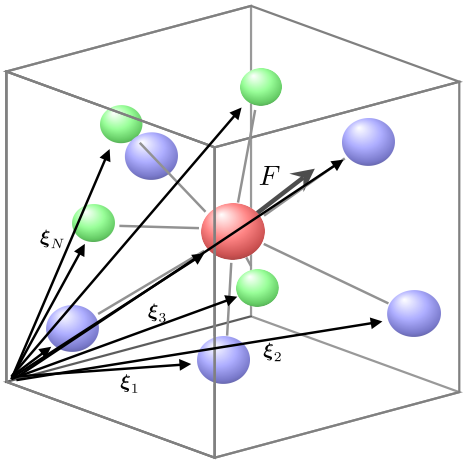

Fig. 2. A prototypical tasks that requires equivariant neural networks is learning from the 3D configuration of an atomic system, represented by the position vectors $(\xi_1, \ldots, \xi_N)$ of the nuclei. This problem already involves three distinct symmetries: translation, rotation, and exchange of identical atoms.

2.1. Computation as a Graph. According to the modern usage of the term, an ANN can be almost any large scale computation represented as a directed acyclic graph (DAG) of elementary operations executed in a massively parallel fashion. Most computational models of the brain certainly fit this description, but in the ANN field, our notion of what the elementary operations, i.e., "neurons," can be greatly expanded over the years. Moreover, representing computation as a graph is not unique to the neural networks. Ultimately, any algebraic expression, and by extension, any function in a computer program can be represented as a DAG, where the edges dictate how the output of each operation feeds into others. The computation graph makes the dependencies between the operations explicit, hence it is a natural starting point for reasoning about different ways of executing the program and optimizing it. Computation graphs are widely used as so-called *intermediate representations* of modern compilers, with the compiler applying a multitude of different transformations to the graph in an effort to improve the efficiency of the generated code (8). At a higher level of granularity, in supercomputers computation graphs are used to optimize the allocation of data and computation to different processors and nodes (9).

Modern neural network libraries such as PyTorch (10), TensorFlow (11) and JAX (12) operate similarly to compilers in the sense that they translate code written in a language like Python into a computation graph. In the case of neural networks, however, the computation graph serves an additional important function as well, which is to allow the software to efficiently compute gradients in the network's backpropagation phase. Here, we come to the second defining feature of modern neural networks, which is that their parameters are typically scattered throughout the network. These parameters are trained in an iterative fashion by computing some overall measure of loss for the entire network relative to training data, and updating the weights by gradient descent. Backpropagation is the process of computing each of the necessary gradients by reversing the flow of information in the computation graph to flow from the final loss through intermediate nodes toward the inputs, and updating the network's parameters along the way. The critical requirement for backpropagation to work is that every individual operation in the network needs to be differentiable.

2.2. Invariance and Equivariance. In line with the above general description of ANNs, a simple feed-forward neural network can be thought of as just a function f mapping inputs $\mathbf{x}$ to outputs $\mathbf{y}$. Consider the case that there is a symmetry group G acting on the inputs of the network, transforming

$$\mathbf{x} \mapsto T_g(\mathbf{x})$$

for any group element $g \in G$. For simplicity, in this paper, we only consider *linear* actions, i.e., we assume that the map T_g corresponding to each group element is linear. The action of translations, rotations, and permutations on atomic configurations mentioned above certainly fulfill this criterion.

The neural network f is said to be **invariant** to G if

$$f(T_g(\mathbf{x})) = f(\mathbf{x})$$

for any $g \in G$. Equivariance is a more general situation than invariance, where both the inputs and the output of a network are allowed to transform under the action of G, but do so in a coordinated way: Given two separate actions $\{T_g\}$ and $\{T'_g\}$ of G on $\mathbf{x}$ and $\mathbf{y}$, we say that f is **equivariant** to G if

$$f(T_g(\mathbf{x})) = T'_g(f(\mathbf{x}))$$

for any group element.

In our running example of learning from atomic configurations, translation invariance is usually relatively easy to implement, simply by using relative position vectors. Permutations are a more involved topic that we will come to later in the paper. Thus, for now we concentrate on just the action of the group SO(3) of the three-dimensional rotations

$$T_{\mathbf{R}}\colon (\xi_1, \ldots, \xi_N) \mapsto (\boldsymbol{R}\xi_1, \ldots, \boldsymbol{R}\xi_N) \qquad \boldsymbol{R} \in \mathrm{SO}(3) \qquad \textbf{[2]}$$

on the atomic position vectors.* Any algorithm that learns a scalar property of a molecular system must be invariant to this action. Equivariance comes into play when we are tasked with learning not just a scalar quantity, but a vector, say a force $\boldsymbol{F}$ from $(\xi_1, \ldots, \xi_N)$. If we rotate the configuration as in Eq. **2**, then the force $\boldsymbol{F}$ must rotate the same way, $\boldsymbol{F} \mapsto \boldsymbol{RF}$, so this type of problem calls for an equivariant neural network. Indeed, force field learning was one of the first areas where equivariant neural networks became popular (13–18).

2.3. Invariant Networks Are Equivariant on the Inside. A second reason why equivariance is important is that invariant networks are often built from equivariant components. At first sight this might seem counterintuitive. Many earlier machine learning attempts approached the invariance problem by computing fixed invariant descriptors of the

*The literature is approximately evenly divided between focusing on the group SO(3) v.s. O(3). These two groups are closely related, the only difference between them being that O(3) also includes reflections. In this paper, for simplicity, we only discuss the SO(3) case, in the understanding that with small modifications, all results generalize to O(3) as well.

Downloaded from https://www.pnas.org by 148.252.133.20 on April 23, 2026 from IP address 148.252.133.20.

inputs, and then applying some generic learning algorithm. A good example of such an invariant descriptor for representing atomic environments is the matrix of dot products between position vectors, sometimes referred to as the *Weyl matrix* (19):

$$W = \begin{bmatrix} \xi_1 \cdot \xi_1 & \xi_1 \cdot \xi_2 & \cdots & \xi_1 \cdot \xi_N \\ \xi_2 \cdot \xi_1 & \xi_2 \cdot \xi_2 & \cdots & \xi_2 \cdot \xi_N \\ \vdots & \vdots & \ddots & \vdots \\ \xi_N \cdot \xi_1 & \xi_N \cdot \xi_2 & \cdots & \xi_N \cdot \xi_N \end{bmatrix}. \quad [3]$$

Clearly, W is invariant under rotations, and so is any learning algorithm that uses the elements of W as input features. Some other classes of invariant descriptors that have proved effective in molecular problems include the Behler–Parrinello descriptors (20), the so-called Coulomb matrix (21), bispectral coefficients, SOAP (22), Moment Tensor Potentials (23), and the Atomic Cluster Expansion (24). Neural network approaches based on such invariant descriptors include (25–28).

The potential drawback of the invariant based neural networks is that because they factor out the action of the symmetry group right at the outset, they miss the opportunity to take into account group-action relative information about the interaction between parts of the system in higher layers. This runs counter to the general neural networks philosophy of learning increasingly complex (or abstract) representations of the data layer by layer (29).

A good analogy is how scalar, vector, and tensor quantities combine in actual physical laws to ultimately give a scalar result. Take, for example, the equation

$$\left(-\nabla p - \rho \boldsymbol{u} \cdot \nabla \boldsymbol{u} + \mu \nabla^2 \boldsymbol{u} + \frac{1}{3}\mu \nabla(\nabla \cdot \boldsymbol{u}) + \rho \boldsymbol{g}\right) \cdot \boldsymbol{n} = 0$$

describing the condition for constant flow velocity along direction $\boldsymbol{n}$ in a viscous fluid. The left hand side of this equation involves a sequence of equivariant operations that finally reduce to an invariant output. In principle, by carefully choosing special coordinate systems we could rewrite this equation as a complicated combination of scalar equations, but this certainly would not be very parsimonious or efficient. In a similar vein, group invariant neural networks are often composed of a sequence of equivariant layers followed by a single layer at the top that symmetrizes over the action of the group.

2.4. Equivariance at the Level of "Neurons". Constructing invariant and equivariant ANNs requires making each elementary component of the network, the titular "neurons," equivariant to the symmetry group. In mathematical terms, this means defining appropriate differentiable operations $\phi\colon (\mathbf{x}_1, \ldots, \mathbf{x}_k) \mapsto \mathbf{y}$ such that for some specific, predefined combination of group actions

$$T_g^{(1)}\colon \mathbf{x}_1 \mapsto \mathbf{x}_1', \quad \ldots\ldots, \quad T_g^{(k)}\colon \mathbf{x}_k \mapsto \mathbf{x}_k'$$

on the inputs of the neuron and a predefined group action $T_g^{\text{out}}\colon \mathbf{y} \mapsto \mathbf{y}'$ on its output, the equivariance condition

$$\phi\left(T_g^{(1)}(\mathbf{x}_1), \ldots, T_g^{(k)}(\mathbf{x}_k)\right) = T_g^{\text{out}}(\phi(\mathbf{x}_1, \ldots, \mathbf{x}_k)) \quad [4]$$

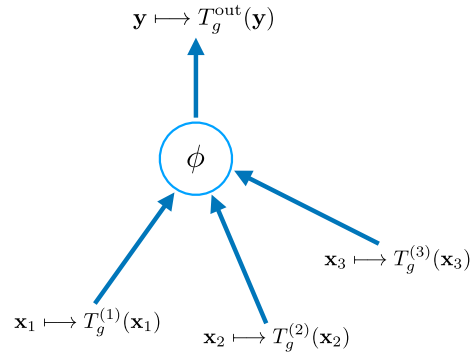

Fig. 3. An equivariant neuron ϕ has the property that if each of its inputs $\mathbf{x}_i$ is transformed according to some corresponding predefined action $T^{(i)}$ of a group G, then its output transforms according to some other action T^{out} of the same group. Any neural network composed exclusively of such neurons is overall equivariant.

is satisfied (Fig. 3). Any network built out of such equivariant building blocks is guaranteed to be equivariant overall. Moreover, the backpropagation process is also automatically equivariant: Letting $\frac{\partial \ell}{\partial y}$ denote the gradient of the loss with respect to the output of ϕ and $\frac{\partial \ell}{\partial x_i}$ the computed gradient with respect to each of the inputs,[†] if $\frac{\partial \ell}{\partial y}$ is transformed as $\frac{\partial \ell}{\partial y} \mapsto T_g^{\text{out}}(\frac{\partial \ell}{\partial y})$, then each $\frac{\partial \ell}{\partial x_i}$ will change as $\frac{\partial \ell}{\partial x_i} \mapsto T^{(i)}(\frac{\partial \ell}{\partial x_i})$.

In general, we call objects transforming according to specific representations of a group G-**covariant**. Hence, a G-equivariant network is one in which every intermediate variable is G-covariant. We emphasize the generality of this framework in that it applies not just to SO(3), but *any* group of symmetries across a wide range of applications. In the next two sections, we derive the rules for constructing equivariant neurons for any symmetry, as long as the corresponding group is compact.

3. Decomposing Group Actions into Irreps

Before delving into the theory of equivariant networks, an important thing to understand is that a given group can have many different actions, and the way to enumerate them systematically is to consider the group's so-called *irreducible representations*. Here, we illustrate these concepts on the example of SO(3) and how it acts on spherical harmonics.

Recall that the spherical harmonics $\{Y_\ell^m\}$ are complex valued functions defined on the unit sphere S^2. In fact, they are a unitary basis for the space $L_2(S^2)$ of all square integrable functions on S^2. The primary index $\ell = 0, 1, 2, \ldots$ plays a role similar to "frequency" in the sense that it captures how many times Y_ℓ^m oscillates as we move around the circumference of the sphere, whereas for any given value of ℓ, the secondary index ranges from $-\ell$ to ℓ and corresponds to the orientation of Y_ℓ^m. Spherical harmonics appear in many contexts in Physics and Chemistry, including the solution of Schrödinger's equation for the Hydrogen

[†] In case of vector valued variables $x = ([x]_1, \ldots, [x]_n)$, we take the liberty of using $\frac{\partial \ell}{\partial x}$ to denote the gradient *vector* $\frac{\partial \ell}{\partial x} = (\frac{\partial \ell}{\partial [x]_1}, \ldots, \frac{\partial \ell}{\partial [x]_n})$.

Downloaded from https://www.pnas.org by 148.252.133.20 on April 23, 2026 from IP address 148.252.133.20.

atom: The familiar s, p, d, f, etc., atomic orbitals correspond directly to the $\ell = 0, 1, 2, \ldots$ spherical harmonics.

What is important for our current purposes is that under a rotation $\boldsymbol{R}$, the spherical harmonics transform as

$$Y_\ell^m(\boldsymbol{R}\,\boldsymbol{r}) = \sum_{m'=-\ell}^{\ell} [D^\ell(\boldsymbol{R})]^*_{m,m'}\, Y_\ell^{m'}(\boldsymbol{r}), \qquad \textbf{[5]}$$

where the $D^\ell(\boldsymbol{R}) \in \mathbb{C}^{(2\ell+1)\times(2\ell+1)}$ matrices are the so-called Wigner D-matrices. Clearly, following one rotation $\boldsymbol{R}_1$ by a second rotation $\boldsymbol{R}_2$ is equivalent to the single rotation $\boldsymbol{R}_2\boldsymbol{R}_1$, hence the Wigner D-matrices satisfy

$$D^\ell(\boldsymbol{R}_2\boldsymbol{R}_1) = D^\ell(\boldsymbol{R}_2)\, D^\ell(\boldsymbol{R}_1). \qquad \textbf{[6]}$$

Now consider the spherical harmonic expansion

$$f(\boldsymbol{r}) = \sum_{\ell=0}^{\infty} \sum_{m=-\ell}^{\ell} c_\ell^m\, Y_\ell^m(\boldsymbol{r})$$

of a function on the sphere and the analogous expansion

$$f'(\boldsymbol{r}) = \sum_{\ell=0}^{\infty} \sum_{m=-\ell}^{\ell} c'^m_\ell\, Y_\ell^m(\boldsymbol{r})$$

of its rotated version $f'(\boldsymbol{r}) = f(\boldsymbol{R}^{-1}\boldsymbol{r})$. Collecting each set of expansion coefficients corresponding to the same ℓ in a vector $\boldsymbol{c}_\ell$, [**5**] tells us that the rotated expansion coefficients can be computed from the original ones just by multiplication by the appropriate D-matrix:

$$\boldsymbol{c}'_\ell = D^\ell(\boldsymbol{R})\, \boldsymbol{c}_\ell. \qquad \textbf{[7]}$$

Abstracting away from spherical harmonics, what this equation does is define an infinite sequence of group actions $T^0_{\mathbf{R}}(\mathbf{v}) := D^0(\boldsymbol{R})\mathbf{v}$, $T^1_{\mathbf{R}}(\mathbf{v}) := D^1(\boldsymbol{R})\mathbf{v}$, ... one for each possible value of ℓ. In principle, any one of this infinite sequence of actions is a possible choice for the inputs or outputs of our equivariant neuron [**4**].

In fact, the sequence of actions defined by Eq. **7** is quite special: In a well-defined sense, the spherical harmonic expansion is the natural generalization of the Fourier series from the circle to the sphere, so the different $\{\mathbf{v}_\ell\}$ vectors give us information about the different "Fourier modes" of f (30). Hence it is quite a natural idea to use exactly these actions as a basis for building SO(3)-equivariant neural networks, with ℓ ranging from 0 to some $\ell_{\max}$.

3.1. Representations. *Group representations* appear in the theory of equivariant neural networks as a way of systematizing the type of Fourier analysis described above, and generalizing it to *any* compact group, without having to explicitly derive the analogs of spherical harmonics and Wigner D-matrices separately in each case.

Formally, a (finite dimensional) representation of a group G is any system of d-dimensional matrices $\{\rho(g) \in \mathbb{C}^{d\times d}\}_{g\in G}$ satisfying

$$\rho(g_2 g_1) = \rho(g_2)\,\rho(g_1) \qquad g_1, g_2 \in G, \qquad \textbf{[8]}$$

just like the Wigner D-matrices satisfy [**6**] for SO(3) (31, 32). Similarly to the D-matrices, representation matrices, in general, are complex valued, and any representation has a corresponding group action on d-dimensional vectors, namely $\mathbf{x} \longmapsto T_g(\mathbf{x}) = \rho(g)\,\mathbf{x}$.[‡] Hence, we could really have phrased our definition [**4**] of what it means for a neuron to be equivariant purely in terms of representations rather than the somewhat vaguer concept of group actions.

A given group can have many different representations, related to each other in various ways. For example, given a d-dimensional unitary matrix T, $\rho'(g) = T^\dagger \rho(g)\, T$ is also a representation of G, but it is hardly any different than ρ, so we call such pairs of representations *equivalent*.[§] It might also happen that after conjugation by T, ρ decomposes into a direct sum of smaller representations $\rho_1, \ldots, \rho_k$, i.e.,

$$\rho(g) = T^\dagger \Big[\bigoplus_i \rho_i(g)\Big] T = T^\dagger \begin{bmatrix} \rho_1(g) & & \\ & \ddots & \\ & & \rho_k(g) \end{bmatrix} T. \qquad \textbf{[9]}$$

In this case, the behavior of ρ can essentially be reduced to studying $\rho_1, \ldots, \rho_k$ separately, so representation theory is mostly focused on representations that cannot be decomposed in this fashion, called *irreducible representations*, or just *irreps* for short. Irreps are like the basic LEGO® bricks of representation theory that other representations are built out of.

The crucial and remarkably general result from representation theory that equivariant neural networks exploit again and again in different ways, is the so-called Peter–Weyl Theorem, stating that any finite dimensional representation of a compact group G is reducible to a direct sum of irreps of G, as in Eq. **9**.[¶] Moreover, this decomposition is essentially unique. Another important result is that a compact group always has a countable (or in the case of finite groups, finite) set of inequivalent irreps.

In the case of SO(3), the irreps are actually just the Wigner D-matrices. For other groups they might be some other sequence of matrix valued functions described in the Mathematics literature.

Regardless of the specifics, the Peter–Weyl theorem strongly suggests that, just like in the spherical harmonic series, equivariant operations should be decomposed according to the irreps of the underlying group. Thus, we can narrow our definition of an equivariant neuron to the following.

Definition 1. Let G be a compact group and $\rho_1, \ldots, \rho_k, \rho_{\text{out}}$ be a sequence of irreps of G. Consider a neural network operation (informally "neuron") ϕ: $(\mathbf{x}_1, \ldots, \mathbf{x}_k) \mapsto \mathbf{y}$ where $\mathbf{x}_1, \ldots, \mathbf{x}_k, \mathbf{y}$ are vectors matching the dimensionalities of

[‡]The correspondence between representations, group actions, and the vector spaces that the actions are defined on is so tight that in the Mathematics literature the word *representation* is often used to refer to the vector space V that $T_g: V \to V$ acts on, rather than the matrix valued function ρ that defines the action. To keep our exposition as straightforward as possible, in this article, we only use *representation* to refer to ρ. However, it is important to bear in mind that in more abstract treatments of the subject, *representation* can refer to ρ or V, almost interchangeably.

[§]Here, M denotes the Hermitian conjugate (conjugate transpose) of the matrix M.

[¶]The analogous result for finite groups is called Maschke's Theorem.

Downloaded from https://www.pnas.org by 148.252.133.20 on April 23, 2026 from IP address 148.252.133.20.

$\rho_1, \ldots, \rho_k, \rho^{\text{out}}$. Such a neuron is said to be **equivariant** to G relative to the above sequence of irreps if

$$\phi\left(\rho_1(g)\mathbf{x}_1, \ldots, \rho_k(g)\mathbf{x}_k\right) = \rho_{\text{out}}(g)\,\phi(\mathbf{x}_1, \ldots, \mathbf{x}_k) \qquad \textbf{[10]}$$

for all group elements $g \in G$.

Before describing some of the basic types of equivariant neurons used in ANNs, some general comments are in order. First, the Peter–Weyl theorem and many other aspects of our exposition crucially rely on G being a *compact* group. In practice, this encompasses:

1. Continuous groups called *Lie groups* that are compact as a manifold, such as O(3) and SO(3).
2. Finite groups, such as the so-called symmetric group $\mathbb{S}_n$ of permutations.

However, there are also various neural networks described in the literature that are equivariant to noncompact groups. Some of the best known examples are:

1. Euclidean group equivariant point cloud networks that are jointly invariant or equivariant to translations and rotations (33–35). This case is obviously important for physical systems, but, as we mentioned above, translation equivariance is often handled simply by using relative position vectors, effectively reducing these networks to SO(n) or O(n)–equivariant architectures.
2. Euclidean group equivariant networks for data on regular meshes for e.g., 3D imaging. Such so-called *steerable* networks handle the interactions between rotations and translations in an altogether different way using induced representations (2, 36, 37). This topic is unfortunately beyond the scope of the present article.
3. Architectures that are equivariant to inherently noncompact groups like the Lorentz group (38), but skirt the compactness issue by restricting themselves to specific finite dimensional irreps and particular equivariant operations between them.

Another issue worth mentioning is why the representations and the vectors that they act on are assumed to be complex valued. Indeed, representation theory does study representations over various other fields as well, including the real numbers, and even finite fields. However, these cases tend to be much more complicated than representations over $\mathbb{C}$ and may lack analogs of, for example, the Peter–Weyl theorem. Given that complex numbers are broadly supported by standard neural network software, it is generally easiest to build equivariant ANNs over complex valued variables.

4. Group Equivariant Neurons

As we saw in Section 2, classical convolutional neural networks involve two basic operations:

1. The convolution operation itself that matches the filters against the input image:

$$\sum_{x'=1}^{n}\sum_{y'=1}^{n} w(x', y')\, h(x+x', y+y'). \qquad \textbf{[11]}$$

This operation is linear in h.

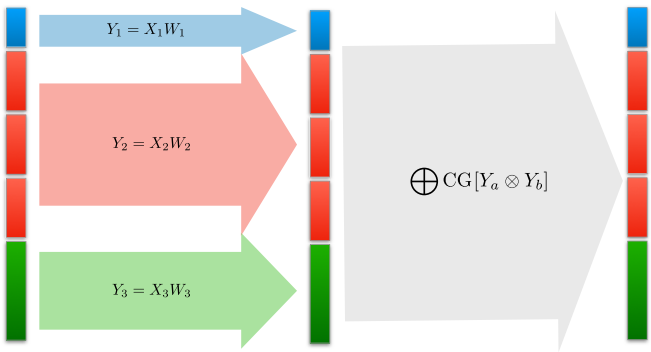

Fig. 4. Equivariant neural networks often operate in "Fourier space," where all variables are decomposed according to the irreps of the underlying symmetry group. An equivariant convolution cannot mix information across different irreps, and, according to *Theorem* 1, it just reduces to multiplying each Fourier component or "part" by a separate learnable matrix. The nonlinearities, on the other hand, typically do need to mix different components. Clebsch–Gordan nonlinearities, in particular, can effectively serve this purpose without having to transform back to the time domain.

2. A nonlinear activation function such as the ReLU.

Both these operations are equivariant to the problem's symmetry group, which is the two-dimensional translation group. We are finally in a position to find analogs of these operations for any compact group G.

4.1. Generalized Convolutions. The key to generalizing convolutional neurons to compact groups, which also shows the importance of decomposing equivariant objects according to the irreps of the group, is a basic result from representation theory called Schur's Lemma:

Proposition 1 [Schur's Lemma, (31) pp. 13]. *Let ρ_1 and ρ_2 be two members of $\mathcal{R}_G$, the complete set of inequivalent irreducible representations of a compact group G over $\mathbb{C}$. Assume that ϕ is a linear function that is equivariant with respect to ρ_1 and ρ_2, i.e.,*

$$\phi(\rho_1(g)\boldsymbol{u}) = \rho_2(g)\,\phi(\boldsymbol{u})$$

for all $g \in G$ and all vectors $\boldsymbol{u}$. Then,

1. *If $\rho_1 \not\cong \rho_2$, then ϕ can only be the zero map $\phi(\boldsymbol{u}) = 0$.*
2. *If $\rho_1 = \rho_2$, then ϕ is a multiple of the identity map.*

Schur's Lemma implies that an equivariant linear neuron cannot mix information from "channels" corresponding to different irreps. This is a striking result similar to e.g., the rule in quantum mechanics that an equation expressing a physical law cannot linearly combine state vectors corresponding to different values of total angular momentum. The constraints that Schur's Lemma puts on equivariant neurons were made explicit in ref. 39, and can be paraphrased as follows.

Theorem 1. *Let $\rho_1, \ldots, \rho_k$ be k distinct, mutually inequivalent irreps of a compact group G of dimensions $d_1, \ldots, d_k$. Let ϕ be a linear neuron that takes as input $\tau_1, \ldots, \tau_k$ vectors transforming according to $\rho_1, \ldots, \rho_k$. Let us collect these vectors in a sequence of matrices*

$$X_1 \in \mathbb{C}^{d_1 \times \tau_1}, \quad \ldots\ldots \quad X_k \in \mathbb{C}^{d_k \times \tau_k}.$$

Downloaded from https://www.pnas.org by 148.252.133.20 on April 23, 2026 from IP address 148.252.133.20.

Assume that ϕ outputs $\tau'_1, \ldots, \tau'_k$ vectors transforming according to the same irreps collected in a similar sequence of matrices $Y_1 \in \mathbb{C}^{d_1 \times \tau'_1}, \ldots, Y_k \in \mathbb{C}^{d_k \times \tau'_k}$. Then ϕ is equivariant if and only if it can be written in the form

$$Y_1 = X_1 W_1, \quad \ldots\ldots \quad Y_k = X_k W_k$$

for some sequence of matrices $\{W_i \in \mathbb{C}^{\tau_i \times \tau'_i}\}$. In the context of generalized convolutional neurons, these matrices serve are the learnable "weights".

Theorem 1 gives a complete characterization of what form G–equivariant linear neurons can take (Fig. 4). It is particularly striking that it reduces convolution on G, a seemingly challenging group theoretical problem, to just a sequence of matrix multiplications, which is exactly the type of dense linear algebra operation that deep learning software and hardware, including graphics processing units, are optimized for.

The price to pay for this dramatic simplification is that the inputs and outputs of ϕ must be expressed in terms of the irreps of the group. From a mathematical point of view, this process is nothing but the natural generalization of Fourier analysis to groups, which has a rich literature in its own right, including some remarkable and relatively recent results on generalized Fast Fourier Transform algorithms for rotation groups, symmetric groups, and some other groups of practical interest (40, 41). Some equivariant neural network models, such as those used in 3D imaging, directly use such generalized FFTs (3).

In Chemistry and Physics the situation is different to computer vision and imaging in that expressing physical quantities in terms of the irreps of SO(3) is so natural that, more often that not, beyond taking the spherical harmonic expansion of (relative) position vectors in the network's input layer, no further forward or backward Fourier transformation is needed in the network at all. This is also why equivariant networks can be more interpretable than networks that do not properly take the problem's symmetry groups into account.

4.2. Equivariant Nonlinearities. Composing linear functions with other linear functions can only yield a function that is itself linear. Hence, to reach any nontrivial level of expressivity, neural networks must also involve some kind of nonlinear operations. In the case of equivariant networks, there are two additional reasons why we must introduce nonlinearities:

- As we have just seen, equivariant linear operations do not allow variables transforming according to different irreps to interact. If we built a network exclusively out of the types of neurons prescribed by *Theorem* 1, we would end up with k completely independent networks that do not communicate with each other.
- A remarkable feature of *Theorem* 1 is that apart from the dimensionalities of the irreps $d_1, \ldots, d_k$, it does not reflect any structural properties of the group G. To build truly powerful equivariant networks, we need some additional equivariant components that encode G more directly.

Historically, ANNs have used a variety of different nonlinear activation functions, such as the hyperbolic tangent function $\sigma_{\tanh}(z) = \tanh(z)$ or the sigmoid $\sigma_{\text{sigmoid}}(z) = 1/(1+e^{-z})$, but interestingly most ANNs today just use the simple ReLU $\sigma_{\text{ReLU}}(z) = \max(0, z)$. Unfortunately, none of these choices are suitable for use in equivariant networks in the direct sense, because as soon as we apply for example a ReLU function to each component $[\boldsymbol{c}_\ell]_m$ of a spherical harmonic expansion vector, it breaks equivariance. Two general strategies have evolved to address this problem.

4.2.1. Real space nonlinearities. The first strategy is to follow every equivariant linear layer with a generalized inverse Fourier transformation that reexpresses the output of the linear layer as a function on G, or a so-called *homogeneous space* of G. A classic example of a homogeneous space is the sphere S^2, which is a homogeneous space of SO(3). The way that groups act on themselves and their homogeneous spaces is by a translation action, for example, if $f: S^2 \to \mathbb{R}$ then under a rotation $\boldsymbol{R}$, f transforms to $f'(\boldsymbol{r}) = f(\boldsymbol{R}^{-1}\boldsymbol{r})$. This implies that we can freely apply any nonlinearity to f in the pointwise sense, without the risk of breaking equivariance. Before the next linear operation, however, we must transform f back to the space of irreps with a generalized Fourier transform.

The obvious drawback of this strategy is the cost of repeated forward and backward Fourier transforms, which can quickly become the computational bottleneck. Nonetheless, this approach is often used in imaging applications, where nondifferentiable nonlinearities like the ReLU can have innate advantages over smoother activation functions.

4.2.2. Clebsch–Gordan nonlinearities. The second strategy is to try and define the nonlinearity itself in Fourier space. In general, this is the approach that is better suited to Physics and Chemistry, especially if the nonlinearities are of polynomial form. After all, most laws of nature are relatively low-order polynomial or rational equations.

Unfortunately, given a G-covariant vector $\mathbf{u}$, individually raising the components of $\mathbf{u}$ to some power does not lead to an equivariant operation. Instead, we need to look to *tensor* (Kronecker) products. If $\mathbf{u} \in \mathbb{C}^{d_1}$ and $\mathbf{v} \in \mathbb{C}^{d_2}$ are two vectors transforming according to irreps ρ and ρ' of a compact group G, then their tensor product

$$\mathbf{u} \otimes \mathbf{v} \in \mathbb{C}^{d_1 \times d_2} \qquad [\mathbf{u} \otimes \mathbf{v}]_{id_2+j} := [\mathbf{u}]_i\, [\mathbf{v}]_j$$

transforms as

$$\mathbf{u} \otimes \mathbf{v} \overset{g}{\longmapsto} (\rho(g) \otimes \rho'(g))\,(\mathbf{u} \otimes \mathbf{v})\,.$$

It is easy to see that $(\rho \otimes \rho')(g) := \rho(g) \otimes \rho'(g)$ is also a representation of G, but, in general, it is not irreducible. However, assuming that G is compact, by the Peter–Weyl theorem we know that it must be reducible into a direct sum of irreps, i.e., there exists a unitary matrix $C_{\rho,\rho'}$ (which is not a function of g) and a sequence of irreps $\rho_1, \ldots, \rho_k$ that block diagonalize $\rho \otimes \rho'$ in the form

$$(\rho \otimes \rho')(g) = C^{\dagger}_{\rho,\rho'} \left(\bigoplus_{i=1}^{k} \rho_i(g) \right) C_{\rho,\rho'} \qquad g \in G. \qquad \textbf{[12]}$$

Downloaded from https://www.pnas.org by 148.252.133.20 on April 23, 2026 from IP address 148.252.133.20.

Using $C^i_{\rho,\rho'}$ to denote the rows of $C_{\rho,\rho'}$ corresponding to the i'th block in Eq. **12** and defining

$$[\mathbf{u}\otimes\mathbf{v}]_i := C^i_{\rho,\rho'}\,(\mathbf{u}\otimes\mathbf{v}), \qquad \textbf{[13]}$$

under the action of G,

$$[\mathbf{u}\otimes\mathbf{v}]_i \xmapsto{g} \rho_i(g)\,[\mathbf{u}\otimes\mathbf{v}]_i,$$

so we have succeeded in creating a quadratic function of the input vectors that is equivariant.

Eq. **12** is called the **Clebsch–Gordan decomposition** (CG-decomposition) of $\rho\otimes\rho'$, and hence [**13**] and its higher order variants are called *Clebsch--Gordan* nonlinearities. The unitary operation

$$\mathbf{u}\otimes_{\mathrm{CG}}\mathbf{v} := \left(C^1_{\rho,\rho'}\,(\mathbf{u}\otimes\mathbf{v}),\ldots,C^k_{\rho,\rho'}\,(\mathbf{u}\otimes\mathbf{v})\right), \qquad \textbf{[14]}$$

where we have one component for each of the terms in the direct sum [**12**], we call the **Clebsch–Gordan product** (or CG-product for short) of **u** and **v**.

The CG-product readily extends to the case where **u** and **v** are not just vectors, but *matrices* with c_1 resp. c_2 columns, i.e., when the inputs have multiple channels. In this case, each output component will be a matrix with c_1c_2 columns. Furthermore, since equivariant neural networks typically carry multiple G–covariant matrices transforming according to different irreps, it also makes sense to further extend [**14**] to the case that the inputs $\boldsymbol{U} = (\mathbf{u}_1,\ldots,\mathbf{u}_{k_1})$ resp. $\boldsymbol{V} = (\mathbf{v}_1,\ldots,\mathbf{v}_{k_2})$ are collections of multiple so-called *G-parts* transforming according to $(\rho_1,\ldots,\rho_{k_1})$ resp. $(\rho'_1,\ldots,\rho'_{k_2})$. The i'th component of the product $\boldsymbol{W} = \boldsymbol{U}\otimes_{\mathrm{CG}}\boldsymbol{V}$ is then

$$[\boldsymbol{U}\otimes_{\mathrm{CG}}\boldsymbol{V}]_i = \bigsqcup_{j_1,j_2} C^i_{\rho_{j_1},\rho_{j_2}}([\boldsymbol{U}]_{j_1}\otimes[\boldsymbol{V}]_{j_2}), \qquad \textbf{[15]}$$

where $\bigsqcup$ stands for stacking matrices together columnwise. The operations Eqs. **14** and **15** are what actually need to be implemented in equivariant network software.

4.2.3. Properties of the CG-decomposition and CG-product. The SO(3) CG-decomposition is well known for its role in the addition of angular momenta in quantum mechanics. Here, the rule for which irreps are featured in the decomposition is particularly simple: $\rho_{\ell_1}\otimes_{\mathrm{CG}}\rho_{\ell_2}$ involves all irreps from $\ell_{\min} = |\ell_1-\ell_2|$ to $\ell_{\max} = \ell_1+\ell_2$, each with multiplicity one.

The general concept of CG-decomposition is not limited to just SO(3), however. Every compact group has its own CG-decomposition, and for most groups of practical interest, which irreps appear in the decomposition, and what the $C_{\rho,\rho'}$ transformation matrices is known. In fact, a general theory called Tannaka–Krein duality (42) shows that the CG-decomposition uniquely defines the structure of the group itself, unfortunately this connection is beyond the scope of our present discussion. The CG-decomposition also plays important roles in complexity theory and quantum computing.[#]

In equivariant neural networks, the CG-product first appeared as a heuristic device in refs. 13 and 14, and in one form or another has been a recurring feature of most architectures since that time (15, 18, 43–45). The CG-product also plays a pivotal role in the bispectrum (22, 46) and in ACE (24). The CG-product is also of central importance in steerable neural networks (36, 37, 47), but in a slightly different role.

All the above is more of a matter of necessity rather than coincidence, since it is possible to show that the ring of polynomial invariants to the action of a compact group is generated by CG-products (38, 48–50). Hence, any kind of polynomial equivariant maps must involve CG-products. This also includes as special cases some very basic operations that might otherwise not be thought of as being related: Multiplying a G-covariant vector **u** by a scalar is the same as taking its CG-product with a vector corresponding to the constant representation ρ_{triv} (in the SO(3) case, the $\ell = 0$ irrep), while computing the norm of **u**, which is the easiest way of symmetrizing over the group, is equivalent to $[\mathbf{u}\otimes_{\mathrm{CG}}\mathbf{u}^*]_{\rho_{\mathrm{triv}}}$. Some recent works also investigated the relationship of higher-order CG-products to so-called *fusion diagrams* appearing in tensor networks, and related potential computational savings (51).

At the practical level, apart from spherical harmonic transforms, which can often be computed off-line as a precomputation step, the CG-product is essentially the only operation in equivariant networks that goes beyond the standard neural networks toolkit. While it can implemented directly, as a tensor product followed by a dense matrix multiplication, from a computational point of view this is generally not advantageous. In response, the community has developed some specialized software libraries for equivariant networks, including E3NN (52) and GElib (53).

5. Equivariance to Permutations

While the previous sections focused on spatial symmetries, a complementary question that is just as important is how to implement the *discrete symmetries* involved in modeling molecular systems. The invariant descriptors mentioned in Section 2 already exhibit a type of duality between these two classes of symmetries. For example, it was proved by Weyl that for any fixed ordering of the particles, the matrix of inner products [**3**] furnishes a system of invariants that is *complete* in the sense that it uniquely defines the positions of the particles up to translation and rotation. However, deriving descriptors from this matrix that are also invariant to permuting the particles yet still maintain the completeness property is just as hard as solving the famed graph isomorphism problem (54). Conversely, representing the environment around a given atom as a continuous "effective particle density" $f\colon \mathbb{R}^3 \to \mathbb{R}$ as it is done in for example SOAP (22) is an easy way to enforce permutation symmetry, but subsequently computing a complete set of rotation invariant descriptors from such a density field f has proved to be challenging.

In neural networks for molecular data, the permutational aspect of the problem is often encoded in a graph, where each atom is either connected to all others within a certain radius, or the edges of the graph correspond to actual chemical bonds. This allows the problem to be conveniently folded into the Graph Neural Networks (GNN) formalism,

[#]Sometimes these connections are obscured by different names being used for the same underlying objects. For example, in the case of finite groups, the CG-decomposition is often called *Kronecker decomposition*. The *Wigner 3-j symbols* are equivalent to the CG-matrices, up to multiplication by a constant and a phase factor, while in some contexts the term *coupling coefficient* is used for the same concept.

Downloaded from https://www.pnas.org by 148.252.133.20 on April 23, 2026 from IP address 148.252.133.20.

which is a separate large branch of the neural networks field.

Most GNNs today follow the so-called *message passing* paradigm (55), where the "neurons" are directly attached to the vertices, and in each layer every vertex collects the outputs of the neighboring vertices from the previous layer. This scheme is relatively easy to combine with the spatially equivariant operations described in the previous sections. In the simplest case, each vertex v_i holds a rotation and translation-covariant vector $\boldsymbol{U}_i$, which in each layer t we update by taking its CG-product with the sum of the corresponding vectors attached to the neighboring vertices, followed by the type of learnable equivariant map prescribed by *Theorem* 1:

$$\boldsymbol{U}_i^{(t)} = \left(\sum_{v_j \in \mathcal{N}(v_i)} \boldsymbol{U}_i^{(t-1)} \otimes_{\mathrm{CG}} \boldsymbol{U}_j^{(t-1)} \right) \times \boldsymbol{W}^{(t)}. \qquad \textbf{[16]}$$

To better capture the geometry, many architectures also involve the spherical harmonic vectors of the relative positions $\xi_i - \xi_j$ in the update rule (15, 18). Indeed, one of the main differences between different types of equivariant point cloud networks is how exactly they combine the spatial equivariance and graph aspects of the problem.

Regardless of these specifics, the basic requirement on message passing layers is that they be equivariant to permutations, i.e., that if we renumber the vertices $\{v_1, \ldots, v_n\}$ of the graph as $\{v_{\sigma(1)}, \ldots, v_{\sigma(n)}\}$ by any permutation σ, the output $\boldsymbol{U}_{\sigma(i)}^{(t)}$ of any given neuron in any layer must be the same as what the output $\boldsymbol{U}_i^{(t)}$ of the corresponding neuron was in the original network. GNNs typically consist of several such layers, followed by a readout layer that sums or averages over the entire graph, ensuring that the final output of the network is fully invariant to renumbering the vertices.

The fact that [**16**] aggregates information from neighboring vertices by summation is a direct consequence of the permutation equivariance requirement: Since summation is a commutative and associate operation, it guarantees that all neighbors of v_i are treated the same way, and hence that the layer as a whole will be permutation equivariant. However, a series of theoretical results as well as empirical observations suggest that using such invariant aggregation rules is too restrictive to capture certain types of structures in graphs (56). The issue is that every time we aggregate messages by summation, information about which part of the aggregate message came from which part of the graph is irretrievably lost. In the image processing context, this would be akin to trying to build a convolutional neural network exclusively out of rotationally symmetric filters.

From the abstract, representation theoretic point of view espoused in this paper, renumbering invariance in GNNs is just another case of a group symmetry, specifically, equivariance to the **symmetric group** $\mathbb{S}_n$ of all *n!* permutations $\sigma\colon \{1, 2, \ldots, n\} \to \{1, 2, \ldots, n\}$. The symmetric group acts on n dimensional vectors (in our case, vectors indexed by the vertices of a graph) by permuting their elements:

$$\boldsymbol{v} \overset{\sigma}{\longmapsto} \boldsymbol{v}' \qquad v'_i = v_{\sigma^{-1}(i)}, \qquad \textbf{[17]}$$

on $n \times n$ matrices (specifically, the adjacency matrix) by permuting their rows and columns:

$$A \overset{\sigma}{\longmapsto} A' \qquad A'_{i,j} = A_{\sigma^{-1}(i),\sigma^{-1}(j)}, \qquad \textbf{[18]}$$

and more generally, on k'th order tensors by permuting each of their dimensions:

$$T \overset{\sigma}{\longmapsto} T' \qquad T'_{i_1,\ldots,i_k} = A_{\sigma^{-1}(i_1),\ldots,\sigma^{-1}(i_k)}. \qquad \textbf{[19]}$$

This sequence of group actions is similar to the action of SO(3) on spherical harmonics of different orders, discussed in Section 3. Indeed, in the Statistics literature, decomposing data according this hierarchy has been used to tease out interactions of different orders in permutational and ranking problems, most brilliantly illustrated in ref. 57.

More broadly, the representation theory of the symmetric group is a rich subject in its own right (58), with its own notion of Clebsch–Gordan products, and even Fast Fourier Transforms (59). One feature that is different from the SO(3) case, however, is that the permutation actions Eqs. **17**–**19** are *not* irreducible. Nonetheless, the Deep Learning community has investigated the problem of equivariant convolutions with respect to these actions. The solution to the first-order case [**17**] was worked out in the influential "Deep Sets" paper (60) and found to have just two learnable parameters per pair of channels. The more general case [**19**] was studied in ref. 61 and it was found that the number of learnable parameters in a convolution operation from a k_1'th order permutation equivariant tensor to a k_2'th order one is given by the Bell number $B(k_1 + k_2)$. Equivalent results derived using a group theoretic approach were derived in ref. 62, but interestingly, it was found that in this case, the algebraic approach does not necessarily translate to computational advantages.

From this perspective, the topology of the graph itself is a type of equivariant side information that effectively reduces the symmetry group of the problem from $\mathbb{S}_n$ to the automorphism group of the graph. Since this may be difficult to account for by purely representation theoretic tools, GNNs that use higher-order permutation actions, often called *subgraph neural networks* or *hypergraph neural networks*, often just instantiate the message passing formalism at the subgraph level (63–65). Recent work on a formalism called *P*-tensors addresses how such subgraphs can communicate with each other so as to guarantee equivariance to permutations at the global level (66). To what extent such more advanced GNN approaches can benefit the representation of point cloud data as well is still an open question.

6. Conclusions

The basic principles behind equivariant neural networks are now well established, and the formalism can be freely combined with other recent advances, such as attention/transformers, various optimization techniques, and generative modeling. However, all this is just one part of building successful neural models for specific physical systems. On the practical side, there are still a host of open

Downloaded from https://www.pnas.org by 148.252.133.20 on April 23, 2026 from IP address 148.252.133.20.

technical challenges in the field, such as building high performance software and hardware frameworks for equivariant architectures, as well as open theoretical questions, such as how to handle noncompact groups. Some other interesting connections that we could not elaborate on in this article are the relationship between equivariant architectures and tensor networks, as well as quantum computing.

Data, Materials, and Software Availability. There are no data underlying this work.

1. T. Cohen, M. Welling, "Group equivariant convolutional networks" in *Proceedings of The 33rd International Conference on Machine Learning, Volume 48 of Proceedings of Machine Learning Research*, M. F. Balcan, K. Q. Weinberger, Eds. (PMLR, New York, New York, USA, 2016), pp. 2990–2999.
2. T. S. Cohen, M Welling, Steerable CNNs. arXiv [Preprint] (2017). https://doi.org/10.48550/arXiv.1612.08498 (Accessed 19 October 2024).
3. T. S. Cohen, M. Geiger, J. Köhler, M. Welling, Spherical CNNs. arXiv [Preprint] (2018). https://doi.org/10.48550/arXiv.1801.10130 (Accessed 19 October 2024).
4. T. Cohen, M. Weiler, B. Kicanaoglu, M. Welling, "Gauge equivariant convolutional networks and the icosahedral CNN" in *Proceedings of the 36th International Conference on Machine Learning*, K. Chaudhuri, R. Salakhutdinov, Eds. (PMLR, 2019).
5. M. M. Bronstein, J. Bruna, T. Cohen, P. Veličković, Geometric deep learning: Grids, groups, graphs, geodesics, and gauges. arXiv [Preprint] (2021). http://arxiv.org/abs/2104.13478 (Accessed 29 October 2024).
6. J. Wood, J. Shawe-Taylor, "Neural networks for invariant pattern recognition" in Proceedings of the ESANN'95 Conference, Brussels (D-Facto Publications, 1995), pp. 253–258.
7. Y. LeCun *et al*., Backpropagation applied to handwritten zip code recognition. *Neural Comput.* **1**, 541–551 (1989).
8. J. Zhao, S. Nagarakatte, M. M. K. Martin, S. Zdancewic, Formalizing the LLVM intermediate representation for verified program transformations. *SIGPLAN Not.* **47**, 427–440 (2020).
9. S. Baruah, "Scheduling DAGs when processor assignments are specified" in *Proceedings of the 31st International Conference on Real-Time Networks and Systems (RTNS)* (ACM, 2020), pp. 1–10
10. A. Paszke *et al*., "Pytorch: An imperative style, high-performance deep learning library" in *Advances in Neural Information Processing Systems 32* (Curran Associates, Inc., 2019), pp. 8024–8035.
11. M. Abadi et al., TensorFlow: Large-scale machine learning on heterogeneous systems. arXiv [Preprint] (2015). https://doi.org/10.48550/arXiv.1603.04467 (Accessed 19 October 2024).
12. J. Bradbury *et al*., JAX: Composable transformations of Python+NumPy programs. GitHub. http://github.com/google/jax. Accessed 19 October 2024.
13. N. Thomas *et al*., Tensor field networks: Rotation- and translation-equivariant neural networks for 3D point clouds. CoRR. arXiv [Preprint] (2018). https://doi.org/10.48550/arXiv.1802.08219 (Accessed 19 October 2024).
14. R. Kondor, N-body networks: A covariant hierarchical neural network architecture for learning atomic potentials. CoRR. arXiv [Preprint] (2018). https://doi.org/10.48550/arXiv.1803.01588 (Accessed 19 October 2024).
15. B. Anderson, T. S. Hy, R. Kondor, "Cormorant: Covariant molecular neural networks" in *Advances in Neural Information Processing Systems 32*, H. Wallach *et al*., Eds. (Curran Associates, Inc., 2019), pp. 14510–14519.
16. S. L. Batzner *et al*., E(3)-equivariant graph neural networks for data-efficient and accurate interatomic potentials. *Nat. Commun.* **13**, 1–14 (2022).
17. A. Musaelian *et al*., Learning local equivariant representations for large-scale atomistic dynamics. *Nat. Commun.* **14**, 579 (2023).
18. I. Batatia, D. P. Kovacs, G. Simm, C. Ortner, G. Csányi, "MACE: Higher order equivariant message passing neural networks for fast and accurate force fields" in *Advances in Neural Information Processing Systems*, S. Koyejo *et al*., Eds. (Curran Associates, Inc., 2022), vol. 35, pp. 11423–11436.
19. H. Weyl, *The Classical Groups* (Princeton University Press, 1946).
20. J. Behler, M. Parrinello, Generalized neural-network representation of high-dimensional potential-energy surfaces. *Phys. Rev. Lett.* **98**, 146401 (2007).
21. M. Rupp, A. Tkatchenko, K. R. Müller, O. A. von Lilienfeld, Fast and accurate modeling of molecular atomization energies with machine learning. *Phys. Rev. Lett.* **108**, 058301 (2012).
22. A. P. Bartók, R. Kondor, G. Csányi, On representing chemical environments. *Phys Rev B* **87**, 184115 (2013).
23. A. V. Shapeev, Moment tensor potentials: A class of systematically improvable interatomic potentials. *Multiscale Model. Simul.* **14**, 1153–1173 (2016).
24. R. Drautz, Atomic cluster expansion: A systematically improvable universal descriptor for interatomic potentials. *Phys. Rev. B* **99**, 014104 (2019).
25. K. T. Schütt, H. E. Sauceda, P. J. Kindermans, A. Tkatchenko, K. R. Müller, Schnet - a deep learning architecture for molecules and materials. *J. Chem. Phys.* **148**, 241722 (2018).
26. J. Klicpera, G. Janek, S. Günnemann, Directional message passing for molecular graphs. arXiv [Preprint] (2020). http://arxiv.org/abs/2003.03123 (Accessed 29 October 2024).
27. O. Unke, K. R. Müller, A machine learning force field for molecular simulations. *Nat. Commun.* **12**, 1–10 (2021).
28. K. T. Schütt, F. Arbabzadah, S. Chmiela, K. R. Müller, A. Tkatchenko, Quantum-chemical insights from deep tensor neural networks. *Nat. Commun.* **8**, 13890 (2017).
29. M. D. Zeiler, R. Fergus, "Visualizing and understanding convolutional networks" in *Computer Vision – ECCV 2014*, D. Fleet, T. Pajdla, B. Schiele, T. Tuytelaars, Eds. (Springer International Publishing, 2013).
30. M. E. Taylor, *Noncommutative Harmonic Analysis, Volume 22 of Mathematical Surveys and Monographs* (American Mathematical Society, Providence, Rhode Island, 1986).
31. J. P. Serre, *Linear Representations of Finite Groups, Volume 42 of Graduate Texts in Mathamatics* (Springer-Verlag, 1977).
32. W. Fulton, J. Harris, *Representation Theory, Graduate Texts in Mathematics* (Springer Verlag, 1991).
33. V. G. Satorras, E. Hoogeboom, M. Welling, "E(n) equivariant graph neural networks" in *Proceedings of the 38th International Conference on Machine Learning (ICML)* (PMLR, 2021).
34. M. Geiger, T. E. Smidt, E3NN: Euclidean neural networks. arXiv [Preprint] (2022). http://arxiv.org/abs/2207.09453 (Accessed 29 October 2024).
35. X. Dong *et al*., Learning local equivariant representations for large-scale atomistic dynamics. *Nat. Commun.* **14**, 1–15 (2023).
36. M. Weiler, M. Geiger, M. Welling, W. Boomsma, T. Cohen, "3D steerable CNNs: Learning rotationally equivariant features in volumetric data" in *Advances in Neural Information Processing Systems* (Neural Information Processing Systems Foundation, 2018), vol. 31, pp. 10381–10392.
37. T. S. Cohen, M. Geiger, M. Weiler, "A general theory of equivariant CNNs on homogeneous spaces" in *Advances in Neural Information Processing Systems 32* (Curran Associates Inc., 2019).
38. A. Bogatskiy *et al*., "Lorentz group equivariant neural network for particle physics" in *Proceedings of the 37th International Conference on Machine Learning, Volume 119 of Proceedings of Machine Learning Research*, H. Daumé III, A. Singh., Eds. (PMLR, 2020), pp. 992–1002.
39. R. Kondor, S. Trivedi, "On the generalization of equivariance and convolution in neural networks to the action of compact groups" in *Proceedings of the 35th International Conference on Machine Learning*, J. Dy, A. Krause, Eds. (PMLR, 2018).
40. D. M. Healy, D. N. Rockmore, S. Moore, "FFTs for the 2-sphere - improvements and variations" (Tech. Rep. PCS-TR96-292, Department of Computer Science, Dartmouth College, 1996).
41. D. Maslen, D. Rockmore, "Generalized FFTs – a survey of some recent results" in *Groups and Computation II, Volume 28 of DIMACS Series in Discrete Mathematics and Theoretical Computer Science*, L. Finkelstein, W. M. Kantor, Eds. (AMS, Providence, RI, 1997), pp. 183–287.
42. A. Joyal, R. Street, "An introduction to Tannaka duality and quantum groups" in *Category Theory, Volume 1488 of Lecture Notes in Mathematics*, A. Carboni, M. C. Pediccio, G. Rosolini, Eds. (Springer, Berlin, Heidelberg, 1991), pp. 413–492.
43. R. Kondor, Z. Lin, S. Trivedi, "Clebsch–Gordan nets: A fully fourier space spherical convolutional neural network" in *Advances in Neural Information Processing Systems 31*, S. Bengio *et al*., Eds. (Curran Associates, Inc., 2016), pp. 10117–10126.
44. Z. Qiao, M. Welborn, A. Anandkumar, F. R. Manby, T. F. Miller, Orbnet: Deep learning for quantum chemistry using symmetry-adapted atomic-orbital features. *J. Chem. Phys.* **153**, 124111 (2020), http://dx.doi.org/10.1063/5.0021955.
45. A. Musaelian *et al*., E(3)-equivariant graph neural networks for data-efficient accurate interatomic potentials. *Nat. Commun.* **13**, 1–12 (2022).
46. R. Kakarala, "Triple correlation on groups," PhD thesis, Department of Mathematics, UC Irvine (1992).
47. L. Lang and M. Weiler, A Wigner-Eckart theorem for group equivariant convolution kernels. arXiv [Preprint] (2021). https://doi.org/10.48550/arXiv.2010.10952 (Accessed 19 October 2024).
48. P. J. Olver, *Classical Invariant Theory, Volume 336 of Graduate Texts in Mathematics* (Springer, New York, 1999).
49. D. Yarotsky, Universal approximations of invariant maps by neural networks. *Constr. Approx.* **55**, 407–474 (2022).
50. H. Lawrence, M. T. Harris, "Learning polynomial problems with $SL(2,\mathbb{R})$-equivariance" in *The Twelfth International Conference on Learning Representations* (2024).
51. Z. Li *et al*., Unifying O(3) equivariant neural networks design with tensor-network formalism. *Mach. Learn. Sci. Technol.* **5**, 025044 (2024).
52. M. Geiger *et al*., The E3NN package (2022). GitHub. https://github.com/e3nn/e3nn. Accessed 19 October 2025.
53. R. Kondor, E. H. Thiede, GElib: A C++/CUDA library for group equivariant operations. GitHub. https://github.com/risi-kondor/GElib. Accessed 19 October 2024.
54. L. Babai, "Graph isomorphism in quasipolynomial time" in *Proceedings of the Forty-Eighth Annual ACM Symposium on Theory of Computing (STOC)* (ACM, 2016), pp. 684–697.
55. J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, G. E. Dahl, "Neural message passing for quantum chemistry" in *Proceedings of the International Conference on Machine Learning (ICML)* (2017).
56. K. Xu, W. Hu, J. Leskovec, S. Jegelka, "How powerful are graph neural networks?" in *International Conference on Learning Representations* (2019).
57. P. Diaconis, *Group Representation in Probability and Statistics, Volume 11 of IMS Lecture Series* (Institute of Mathematical Statistics, 1988).
58. B. E. Sagan, *The Symmetric Group, Graduate Texts in Mathematics* (Springer, 2001).
59. M. Clausen, Fast generalized Fourier transforms. *Theor. Comput. Sci.* **67**, 55–63 (1989).
60. M. Zaheer *et al*., "Deep sets" in *Advances in Neural Information Processing Systems*, I. Guyon, U. von Luxburg, S. Bengio, H. Wallach, Eds. (Curran Associates, 2017), vol. 30.
61. H. Maron, H. Ben-Hamu, N. Shamir, Y. Lipman, Invariant and equivariant graph networks. arXiv [Preprint] (2019). https://doi.org/10.48550/arXiv.1812.09902 (Accessed 19 October 2024).
62. E. H. Thiede, T. Hy, R. Kondor, The general theory of permutation equivarant neural networks and higher order graph variational encoders. arXiv [Preprint] (2020). https://doi.org/10.48550/arXiv.2004.03990 (Accessed 19 October 2024).
63. E. Alsentzer, S. Finlayson, M. Li, M. Zitnik, "Subgraph neural networks" in *Advances in Neural Information Processing Systems (NeurIPS)*, A. Agarwal, D. Belgrave, K. Cho, A. Oh, Eds. (Curran Associates, 2020). vol. 33.
64. B. Bevilacqua *et al*., "Equivariant subgraph aggregation networks" in *International Conference on Learning Representations* (2022).
65. L. Zhao, W. Jin, L. Akoglu, N. Shah, From stars to subgraphs: Uplifting any GNN with local structure awareness. arXiv [Preprint] (2022). https://doi.org/10.48550/arXiv.2110.03753 (Accessed 19 October 2024).
66. A. R. Hands, T. Sun, R. Kondor, "P-tensors: A general framework for higher order message passing in subgraph neural networks" in *International Conference on Artificial Intelligence and Statistics* (PMLR, 2024).

Downloaded from https://www.pnas.org by 148.252.133.20 on April 23, 2026 from IP address 148.252.133.20.